

HƯỚNG DẪN KIẾN TRÚC VÀ XÁC THỰC

API HẠN MỨC TÍN DỤNG

IntegrationHub API · MSSQL dbo.vw_Tin_Dung

Luồng xử lý từng class · JWT RSA private/public key · Token 30 phút · Hướng dẫn tích hợp

TRẠNG THÁI HIỆN TẠI Endpoint lấy hạn mức tín dụng và kiểm tra scope finance.read đã có trong mã nguồn. Dịch vụ cấp token/Identity Provider chưa nằm trong repository; các địa chỉ token trong tài liệu phải được thay bằng cấu hình môi trường thực tế.

Phiên bản 1.0 | 20/08/2026

Mục lục nội dung

1. Phạm vi và cách đọc tài liệu
2. Tổng quan kiến trúc API
3. Trình tự xử lý một request từ class nào đến class nào
4. Giải thích chi tiết từng model và class
5. Hướng dẫn sử dụng API hạn mức tín dụng
6. Kiến trúc xác thực JWT bằng private key/public key
7. Quy trình lấy và tái sử dụng access token
8. Quy tắc hết hạn 30 phút và cấu hình đề xuất
9. Triển khai, vận hành và kiểm thử
10. Phụ lục ví dụ mã nguồn

1. Phạm vi và cách đọc tài liệu

Tài liệu này giải thích endpoint GET /api/v1/finance/credit-limits trong dự án api_core_system. Mục tiêu là giúp người phát triển hiểu cấu trúc nhiều lớp, luồng thực thi, dữ liệu vào/ra, cách truy vấn SQL Server và cách bảo vệ API bằng access token JWT.

KẾT LUẬN NGẮN Client không gọi trực tiếp SQL Server. Client lấy access token từ Authorization Server, gửi token đến IntegrationHub API, API kiểm tra token và scope finance.read, sau đó mới gọi dbo.vw_Tin_Dung bằng câu lệnh SQL có tham số.

1.1 Những gì đã có trong mã nguồn

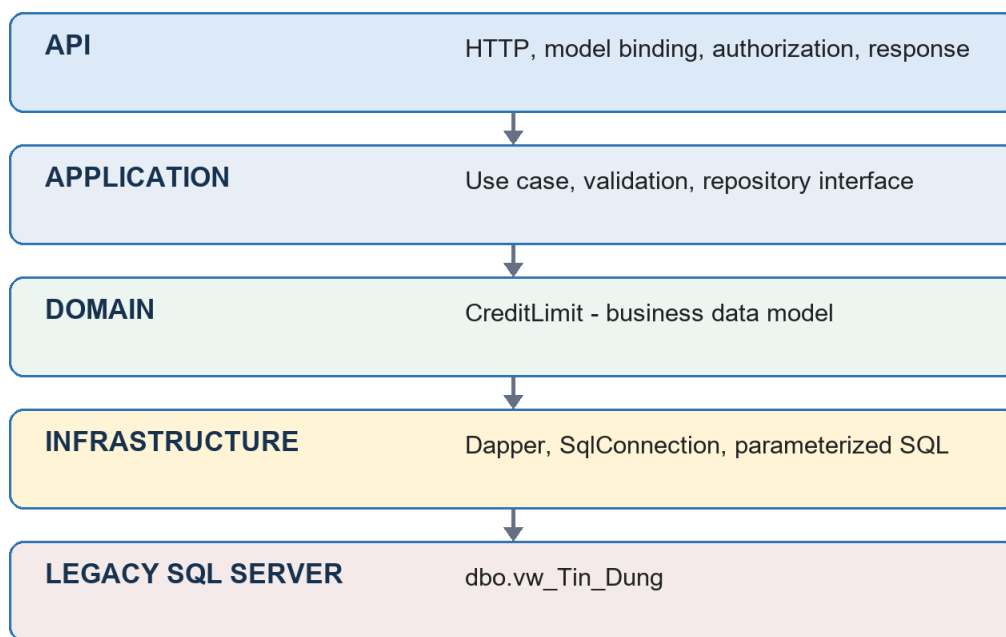
- Endpoint GET /api/v1/finance/credit-limits với hai query parameter ma_dt và ngay_hl.
- Policy authorization finance.read.
- JWT Bearer authentication dựa trên Authentication:Authority và Authentication:Audience.
- Application service kiểm tra dữ liệu đầu vào và chuẩn hóa ngày.
- Repository Dapper truy vấn dbo.vw_Tin_Dung bằng parameter, không ghép chuỗi SQL.
- Domain model CreditLimit và JSON camelCase.

1.2 Những gì chưa có

- Không có endpoint đăng nhập/cấp token trong IntegrationHub API.
- Không có private key trong repository và không nên thêm private key vào API này.
- Authority hiện là giá trị mẫu identity.example.com; cần cấu hình Identity Provider thật.
- Quy tắc token tối đa 30 phút chưa được cường chế độc lập tại API; hiện API tin vào exp do Authority phát hành và cho ClockSkew 1 phút.

2. Tổng quan kiến trúc API

Dự án áp dụng kiến trúc phân lớp. Mỗi lớp chỉ giữ một loại trách nhiệm. Điều này giúp thay đổi database, quy tắc nghiệp vụ hoặc giao thức HTTP mà không làm toàn bộ mã nguồn phụ thuộc lẫn nhau.



Dependency direction: API -> Application <- Infrastructure; Domain contains no database code.

Hình 1. Các lớp chính của use case hạn mức tín dụng

Lớp	Trách nhiệm	Thành phần chính
API	Nhận HTTP, binding tham số, yêu cầu quyền, trả HTTP response	CreditLimitsController, middleware auth
Application	Điều phối use case, validate, định nghĩa cổng repository	CreditLimitService, GetCreditLimitsQuery, ICreditLimitReadRepository
Domain	Mô hình dữ liệu nghiệp vụ độc lập	CreditLimit

Lớp	Trách nhiệm	Thành phần chính
Infrastructure	Kết nối công nghệ bên ngoài và thực thi SQL	CreditLimitReadRepository, SqlServerConnectionFactory
Legacy SQL	Nguồn dữ liệu thực	dbo.vw_Tin_Dung

2.1 Dependency Injection khi khởi động

Program.cs gọi AddApplication() và AddInfrastructure(configuration). Hai extension method đăng ký các class vào DI container:

```
AddScoped<CreditLimitService>();
AddScoped<ICreditLimitReadRepository, CreditLimitReadRepository>();
AddSingleton<ILegacyDbConnectionFactory, SqlServerConnectionFactory>();
```

Khi ASP.NET Core cần tạo CreditLimitsController, DI tự tạo CreditLimitService; khi service cần ICreditLimitReadRepository, DI cấp CreditLimitReadRepository; repository nhận tiếp connection factory và database options. Đây là constructor injection.

3. Trình tự xử lý một request

REQUEST MẪU GET /api/v1/finance/credit-limits?ma_dt=1000001&ngay_hl=2026-08-20 với header Authorization: Bearer <access_token>.

1. **Client gửi HTTPS request:** Query string mang ma_dt và ngay_hl; access token nằm trong header Authorization.
2. **UseAuthentication() chạy:** JwtBearerHandler đọc Bearer token và tải metadata/JWKS từ Authority khi cần.
3. **JWT được kiểm tra:** Chữ ký, issuer, audience và thời gian exp/nbf được kiểm tra. Token sai trả 401 trước khi controller chạy.
4. **UseAuthorization() chạy:** Policy finance.read kiểm tra claim scope. Thiếu scope trả 403.
5. **ASP.NET Core chọn controller:** Route api/v1/finance/credit-limits ánh xạ đến CreditLimitsController.Get().
6. **Model binding:** ma_dt được gán vào partnerCode; ngay_hl được parse thành DateTime? effectiveDate.
7. **Controller tạo query model:** new GetCreditLimitsQuery(partnerCode, effectiveDate).
8. **Controller gọi application service:** CreditLimitService.GetAsync(query, cancellationToken).

9. **Service validate:** Bắt buộc ma_dt, tối đa 20 ký tự, bắt buộc ngay_hl; Trim mã và lấy Date của ngày hiệu lực.
10. **Service gọi repository interface:** ICreditLimitReadRepository.GetAsync(...). Service không biết Dapper hay SQL Server.
11. **Repository mở kết nối:** SqlConnectionFactory tạo SqlConnection từ LegacyDatabase.ConnectionString.
12. **Repository chạy SQL:** Dapper truyền @PartnerCode và @EffectiveDate dưới dạng parameter có kiểu dữ liệu.
13. **Dapper ánh xạ kết quả:** Alias cột SQL được map vào constructor của CreditLimit.
14. **Controller trả response:** Ok(list) tạo HTTP 200; JSON serializer đổi tên property sang camelCase.

```
Client
-> JWT middleware
-> finance.read policy
-> CreditLimitsController.Get
-> CreditLimitService.GetAsync
-> ICreditLimitReadRepository.GetAsync
-> CreditLimitReadRepository.GetAsync
-> SqlConnectionFactory.OpenConnectionAsync
-> dbo.vw_Tin_Dung
-> CreditLimit[] -> HTTP 200 JSON
```

4. Giải thích chi tiết từng model và class

4.1 CreditLimitsController - lớp HTTP

File: src/IntegrationHub.Api/Controllers/CreditLimitsController.cs

- [Route] định nghĩa URL của endpoint.
- [Authorize(Policy = ApiScopes.FinanceRead)] yêu cầu token có scope finance.read.
- [FromQuery(Name = "ma_dt")] và ngay_hl ánh xạ đúng tên parameter mà hệ thống tích hợp gửi.
- Controller chỉ chuyển đổi HTTP thành use case; không viết SQL trong controller.
- ArgumentException từ service được chuyển thành HTTP 400 ValidationProblemDetails.

4.2 GetCreditLimitsQuery - input model

```
public sealed record GetCreditLimitsQuery(
    string? PartnerCode,
    DateTime? EffectiveDate);
```

Đây là dữ liệu đầu vào nội bộ của use case. Dùng record giúp object bất biến theo ý nghĩa: controller tạo một giá trị query rồi truyền xuống service.

4.3 CreditLimitService - quy tắc ứng dụng

Service kiểm tra dữ liệu, tránh để controller hoặc repository gánh quy tắc nghiệp vụ. Nó trim PartnerCode, giới hạn 20 ký tự theo cột Ma_Dt varchar(20), yêu cầu EffectiveDate và chuẩn hóa Value.Date để bỏ phần giờ.

LƯU Ý NGHIỆP VỤ Điều kiện hiện tại chỉ là Ngay_Kt >= ngay_hl. Nó chưa kiểm tra Ngay_Bd <= ngay_hl. Đây là đúng theo câu SQL yêu cầu ban đầu; nếu muốn 'còn hiệu lực tại một ngày', thường cần đồng thời Ngay_Bd <= ngay_hl.

4.4 ICreditLimitReadRepository - abstraction

Interface nằm ở Application, mô tả điều Application cần mà không chỉ ra công nghệ thực hiện. Nhờ đó có thể thay repository MSSQL bằng mock khi test hoặc nguồn dữ liệu khác mà service không đổi.

4.5 CreditLimitReadRepository - SQL adapter

Repository chứa câu SQL cố định, tạo DynamicParameters, mở connection và gọi Dapper QueryAsync<CreditLimit>. Parameterized SQL ngăn ma_dt bị hiểu như một đoạn SQL và giúp SQL Server tái sử dụng execution plan.

```
WHERE Ngay_Kt >= @EffectiveDate
AND Ma_Dt = @PartnerCode
```

4.6 SqlConnectionFactory

Factory đọc connection string từ options, tạo Microsoft.Data.SqlClient.SqlConnection và mở kết nối bất đồng bộ. await using bảo đảm connection được dispose sau khi query hoàn tất hoặc phát sinh lỗi.

4.7 CreditLimit - output/domain model

Cột SQL	Property C# / JSON	Ý nghĩa
Ma_Dt	PartnerCode / partnerCode	Mã đối tượng/khách hàng
Ma_Hd	ContractCode / contractCode	Mã hợp đồng
Tien_Tin_Chap	UnsecuredAmount / unsecuredAmount	Hạn mức tín chấp
Tien_Bao_Lanh	GuaranteeAmount / guaranteeAmount	Hạn mức bảo lãnh
Tien_Cam_Co	CollateralAmount / collateralAmount	Hạn mức cầm cố
Tien_Tin_Chap_Nt	UnsecuredForeignCurrencyAmount	Tín chấp nguyên tệ
Tien_Bao_Lanh_Nt	GuaranteeForeignCurrencyAmount	Bảo lãnh nguyên tệ
Tien_Cam_Co_Nt	CollateralForeignCurrencyAmount	Cầm cố nguyên tệ

Cột SQL	Property C# / JSON	Ý nghĩa
Ngày_Bd	StartDate / startDate	Ngày bắt đầu
Ngày_Kt	EndDate / endDate	Ngày kết thúc

Các cột money được CONVERT(decimal(19,4)) trước khi Dapper map sang decimal để API có kiểu số rõ ràng. JSON serializer được cấu hình camelCase trong Program.cs.

5. Hướng dẫn sử dụng API hạn mức tín dụng

5.1 Thông tin bắt buộc

Thông tin	Giá trị	Bắt buộc
Base URL	Ví dụ https://api.company.vn	Có - theo môi trường
HTTP method	GET	Có
Path	/api/v1/finance/credit-limits	Có
ma_dt	Mã đối tượng, tối đa 20 ký tự	Có
ngay_hl	Ngày ISO yyyy-MM-dd	Có
Authorization	Bearer <access_token>	Có
Scope trong token	finance.read	Có

5.2 Ví dụ cURL

```
curl --request GET \
  'https://api.company.vn/api/v1/finance/credit-limits?ma_dt=1000001&ngay_hl=2026-08-20' \
  --header 'Authorization: Bearer eyJhbGciOiJSUzI1NiIs...' \
  --header 'Accept: application/json'
```

5.3 Ví dụ response 200

```
[
  {
    "partnerCode": "1000001",
    "contractCode": "HD-001",
    "unsecuredAmount": 500000000.0000,
    "guaranteeAmount": 0.0000,
    "collateralAmount": 200000000.0000,
    "unsecuredForeignCurrencyAmount": 0.0000,
    "guaranteeForeignCurrencyAmount": 0.0000,
    "collateralForeignCurrencyAmount": 0.0000,
    "startDate": "2026-01-01T00:00:00",
    "endDate": "2026-12-31T00:00:00"
  }
]
```

KẾT QUẢ RỖNG Nếu không có bản ghi phù hợp, API trả HTTP 200 với mảng rỗng []. Đây không phải lỗi 404.

5.4 Mã trạng thái cần xử lý

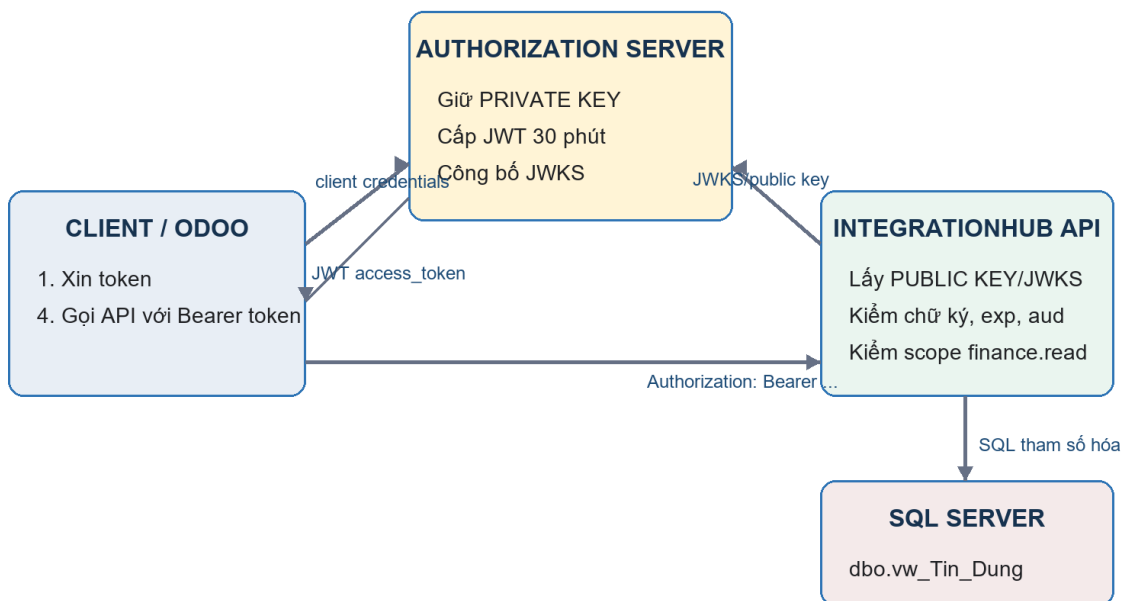
HTTP	Ý nghĩa	Cách xử lý phía client
200	Thành công; có thể là []	Đọc JSON và xử lý danh sách
400	Thiếu/sai ma_dt hoặc ngay_hl	Sửa request; không xin token lại
401	Không có token, token sai chữ ký hoặc hết hạn	Xin token mới rồi retry một lần
403	Token hợp lệ nhưng thiếu finance.read	Cấp đúng scope; xin token mới
500	Lỗi không xử lý trong API	Ghi correlation ID/log, retry có kiểm soát
503	Phụ thuộc chưa sẵn sàng nếu được cấu hình	Backoff; kiểm tra health/readiness

5.5 Ví dụ C# HttpClient

```
using var request = new HttpRequestMessage(
    HttpMethod.Get,
    $"{baseUrl}/api/v1/finance/credit-
limits?ma_dt={Uri.EscapeDataString(maDt)}&ngay_hl={ngayHl:yyyy-MM-dd}");
request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", accessToken);
using var response = await httpClient.SendAsync(request, cancellationToken);
response.EnsureSuccessStatusCode();
```


6. Kiến trúc xác thực JWT bằng private/public key

Private key never leaves the Authorization Server



Hình 2. Luồng cấp token, kiểm tra JWT và truy vấn dữ liệu

6.1 Vai trò của hai loại khóa

Khóa	Nơi lưu	Mục đích	Có được chia sẻ?
Private key	Authorization Server / HSM / Key Vault	Ký JWT access token	Không; tuyệt đối không đưa vào API/client/repository
Public key	Công bố qua JWKS; API cache	Kiểm tra chữ ký JWT	Có; đây không phải bí mật

Cơ chế này thường dùng RS256 hoặc PS256. Authorization Server ký phần header.payload bằng private key. IntegrationHub API dùng public key tương ứng với kid trong JWT để xác nhận token không bị sửa và đúng nơi phát hành.

NGUYÊN TẮC IntegrationHub API không cần và không nên có private signing key nếu nó chỉ là resource server. Nếu private key của Authorization Server bị lộ, kẻ tấn công có thể tự tạo token hợp lệ.

6.2 Claims tối thiểu nên có

Claim	Ví dụ	Mục đích
-------	-------	----------

Claim	Ví dụ	Mục đích
iss	https://identity.company.vn/realms/integration	Định danh nơi cấp token
aud	integration-api	Token dành cho API nào
sub/client_id	odoo-integration	Định danh client gọi
scope	finance.read	Quyền được phép
iat	Unix timestamp	Thời điểm phát hành
nbf	Unix timestamp	Không hợp lệ trước thời điểm này
exp	iat + 1800 giây	Hết hạn sau tối đa 30 phút
jti	UUID ngẫu nhiên	Định danh duy nhất của token

6.3 Kiểm tra trong mã nguồn hiện tại

ScopeAuthorization.AddApiAuthentication() dùng AddJwtBearer. Authority cho middleware biết issuer và địa chỉ discovery; Audience là integration-api. Middleware tự tải OpenID Connect metadata và JWKS public keys, sau đó kiểm tra Bearer token.

```
options.Authority = authority;
options.Audience = audience;
options.RequireHttpsMetadata = true;
options.MapInboundClaims = false;
options.TokenValidationParameters.ClockSkew = TimeSpan.FromMinutes(1);
```

HasScope() đọc tất cả claim scope, tách theo khoảng trắng và so sánh chính xác với finance.read. Token có thể chứa nhiều scope, ví dụ "master.read finance.read".

6.4 Luân chuyển khóa (key rotation)

1. **Bước:** Authorization Server tạo cặp khóa mới và thêm public key mới vào JWKS.
2. **Bước:** Bắt đầu ký token mới bằng private key mới; JWT header mang kid mới.
3. **Bước:** Giữ public key cũ trong JWKS ít nhất bằng thời gian sống token cộng độ lệch đồng hồ.
4. **Bước:** Sau khi mọi token cũ hết hạn, mới loại public key cũ khỏi JWKS.
5. **Bước:** Không đổi khóa đột ngột nếu không muốn toàn bộ token đang dùng bị 401.

7. Quy trình lấy và tái sử dụng access token

7.1 Khuyến nghị: OAuth 2.0 Client Credentials

Luồng này phù hợp cho Odoo/backend gọi backend, không có người dùng tương tác. Client gửi client_id và bí mật xác thực đến token endpoint của Authorization Server. Authorization Server trả access_token có scope finance.read.

ĐỊA CHỈ TOKEN Repository hiện không định nghĩa token endpoint. Hãy đọc discovery document tại {Authority}/.well-known/openid-configuration và lấy trường token_endpoint. Nếu Authority thực tế là Keycloak, đường dẫn thường có dạng {Authority}/protocol/openid-connect/token.

```
curl --request POST '[TOKEN_ENDPOINT]' \
  --header 'Content-Type: application/x-www-form-urlencoded' \
  --data-urlencode 'grant_type=client_credentials' \
  --data-urlencode 'client_id=odoo-integration' \
  --data-urlencode 'client_secret=[CLIENT_SECRET]' \
  --data-urlencode 'scope=finance.read'
```

7.2 Response token điển hình

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsImtpZCI6Ii4uLiJ9...",
  "token_type": "Bearer",
  "expires_in": 1800,
  "scope": "finance.read"
}
```

7.3 Cách lưu và tái sử dụng

1. **Lưu trong bộ nhớ:** Backend giữ access_token và expires_at = thời điểm nhận + expires_in. Không ghi token vào log.
2. **Tái sử dụng:** Dùng cùng token cho nhiều request trong thời gian hợp lệ; không cần xin token cho từng request.
3. **Làm mới sớm:** Khi còn dưới 60 giây, xin token mới trước request tiếp theo để tránh hết hạn giữa đường.
4. **Gặp 401:** Xóa token cache, xin token mới và retry đúng một lần; không retry vô hạn.
5. **Đồng bộ thời gian:** Client, Authorization Server và API phải dùng NTP; sai giờ gây token chưa có hiệu lực hoặc hết hạn sớm.

7.4 Client authentication bằng private_key_jwt (nâng cao)

Nếu không muốn dùng client_secret, mỗi client có thể giữ private key riêng để ký client_assertion khi xin token. Authorization Server lưu public key của client để xác minh. Đây là cặp khóa của client, khác với cặp khóa Authorization Server dùng để ký access token.

```
POST [TOKEN_ENDPOINT]
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
client_id=odoo-integration
scope=finance.read
client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
client_assertion=[JWT_DO_CLIENT_KY_BANG_PRIVATE_KEY_CUA_CLIENT]
```

JWT client_assertion	Giá trị yêu cầu
iss	client_id
sub	client_id
aud	token endpoint chính xác
exp	Ngắn, thường không quá vài phút
jti	Duy nhất để chống replay

PHÂN BIỆT Private key của client chỉ dùng chứng minh danh tính client khi xin token. Private key của Authorization Server dùng ký access token. IntegrationHub API chỉ cần public key/JWKS của Authorization Server.

8. Quy tắc hết hạn 30 phút

8.1 Cấu hình bắt buộc tại Authorization Server

- Access token lifetime = 1800 giây.
- Token phải có iat và exp; exp - iat không lớn hơn 1800 giây.
- Không cấp scope finance.read cho client không được phép.
- Client Credentials thường không cần refresh token; hết hạn thì gọi token endpoint để lấy token mới.

8.2 Cấu hình API để hết hạn đúng thời điểm

Mã hiện tại đặt ClockSkew = 1 phút, nghĩa là middleware có thể chấp nhận token thêm tối đa khoảng một phút quanh mốc thời gian để bù sai lệch đồng hồ. Nếu yêu cầu nghiệp vụ là hết hạn chính xác sau 30 phút, đổi về TimeSpan.Zero và bắt buộc đồng bộ NTP.

```
options.TokenValidationParameters.ClockSkew = TimeSpan.Zero;
```

QUAN TRỌNG Chỉ kiểm tra exp chưa ngăn Authorization Server vô tình cấp token sống 2 giờ. Để bảo vệ kép, API có thể thêm LifetimeValidator kiểm tra exp - iat <= 30 phút. Tuy nhiên cấu hình chuẩn nhất vẫn là đặt lifetime tại Authorization Server.

8.3 Ma trận thời gian

Thời điểm	Trạng thái	Hành động
T0	Client nhận token expires_in=1800	Cache token và expires_at
T0 + 0..28:59	Token dùng bình thường	Tái sử dụng
T0 + 29:00	Vùng làm mới sớm 60 giây	Xin token mới trước request tiếp theo
T0 + 30:00	Token hết hạn	API trả 401 nếu ClockSkew=0
Sau 401	Token cache không còn dùng được	Xin mới và retry một lần

9. Triển khai, vận hành và kiểm thử

9.1 Cấu hình API

```
Authentication__Authority=https://identity.company.vn/realms/integration
Authentication__Audience=integration-api
LegacyDatabase__ConnectionString=Server=...;Database=...;User
ID=integration_api;Password=...;Encrypt=True;TrustServerCertificate=False
```

Không commit connection string, client secret, private key hoặc access token. Dùng secret store của môi trường triển khai. API bắt buộc HTTPS và Authority cũng phải dùng HTTPS.

9.2 Quyền database tối thiểu

```
GRANT SELECT ON OBJECT::dbo.vw_Tin_Dung TO integration_api;
```

Tài khoản integration_api chỉ cần SELECT view. Không cấp db_owner hoặc quyền ghi nếu use case chỉ đọc.

9.3 Bộ test chấp nhận

Kịch bản	Kết quả mong đợi
Không gửi token	401
Token chữ ký sai	401
Token đúng nhưng aud khác	401
Token hết hạn 30 phút	401
Token thiếu finance.read	403
Thiếu ma_dt hoặc ngay_hl	400
ngay_hl sai định dạng	400
ma_dt dài hơn 20 ký tự	400
Không có dữ liệu	200 và []
Có dữ liệu	200 và JSON đúng mapping
ma_dt chứa ký tự SQL injection	Không làm thay đổi câu SQL; chỉ là giá trị parameter

9.4 Logging an toàn

- Ghi request ID/correlation ID, client_id/sub, route, status code và thời gian xử lý.
- Không ghi Authorization header, access token, private key, client secret hoặc connection string.
- Có thể ghi ma_dt theo chính sách dữ liệu nội bộ; cân nhắc masking nếu đây là thông tin nhạy cảm.
- Theo dõi riêng số lượng 401, 403, lỗi kết nối MSSQL và query timeout.

10. Phụ lục ví dụ mã nguồn

10.1 Pseudocode token cache phía client

```

if (cachedToken is null ||.UtcNow >= cachedToken.ExpiresAt - 60 seconds)
{
    cachedToken = await RequestClientCredentialsToken();
}

response = await CallCreditLimitApi(cachedToken.AccessToken);

if (response.StatusCode == 401 && notRetried)
{
    cachedToken = await RequestClientCredentialsToken();
    response = await CallCreditLimitApi(cachedToken.AccessToken);
}

```

10.2 Checklist trước khi go-live

- Authority và Audience đã trở đúng môi trường.
- Authorization Server ký JWT bằng RSA; private key ở HSM/Key Vault hoặc secret store bảo vệ.
- JWKS truy cập được từ API và có quy trình rotation.
- Access token lifetime = 1800 giây; ClockSkew được chọn phù hợp yêu cầu.
- Client được cấp đúng finance.read và không có scope thừa.
- Database user chỉ có SELECT dbo.vw_Tin_Dung.
- HTTPS, NTP, logging redaction và monitoring đã bật.
- Đã chạy toàn bộ test 401/403/400/200 và hết hạn 30 phút.

ĐỀ XUẤT BƯỚC TIẾP THEO Chọn Authorization Server cụ thể (ví dụ Keycloak hoặc dịch vụ OIDC nội bộ), cấu hình realm/client/scope và token lifetime 1800 giây; sau đó cập nhật Authority thật, đặt ClockSkew theo yêu cầu và bổ sung integration test dùng token đã ký bằng key thử nghiệm.

Tài liệu mã nguồn tham chiếu

- src/IntegrationHub.Api/Program.cs
- src/IntegrationHub.Api/Authentication/ScopeAuthorization.cs
- src/IntegrationHub.Api/Authentication/ApiScopes.cs
- src/IntegrationHub.Api/Controllers/CreditLimitsController.cs
- src/IntegrationHub.Application/Modules/Finance/CreditLimits/CreditLimitService.cs
- src/IntegrationHub.Application/Modules/Finance/CreditLimits/GetCreditLimitsQuery.cs
- src/IntegrationHub.Application/Abstractions/ICreditLimitReadRepository.cs
- src/IntegrationHub.Infrastructure/LegacySqlServer/Finance/CreditLimitReadRepository.cs
- src/IntegrationHub.Infrastructure/LegacySqlServer/SqlServerConnectionFactory.cs
- src/IntegrationHub.Domain/Finance/CreditLimit.cs